一、 问题概述

（一）问题描述

本实验基于经典的八皇后问题，使用 python 语言，用逻辑编程的思想解决逻辑约束问题。需在 8*8 的棋盘上，放置八个皇后，并且满足：

1. 任意两个皇后不能在同一横行
2. 任意两个皇后不能在同一纵行
3. 任意两个皇后不能在同一斜线

（二）问题分析

本实验涉及逻辑推理中的逻辑否定约束问题，而通过阅读 kanren 库源码，其更注重正向的逻辑推导，对于逻辑否定并没有定义相应的函数。所以本实验的难点就在于需要使用逻辑编程的方式，通过规则约束来逐步缩小解的搜索空间。同时注意斜线的约束条件包括了正对角线和副对角线两条。

（三）解决问题的思路和想法

逻辑编程的核心是通过定义事实 facts 和关系 relation 声明逻辑关系，再通过逻辑推理引擎 run 来寻找满足所有规则的解。因此解决本问题的思路和想法如下：

1. 使用回溯算法，利用 python 编写递归函数逐行放置皇后，并同时检查是否满足所有的约束条件。如果发现冲突，则回溯到上一步，尝试其他位置。
2. 声明 queen_location 的关系，将得到的所有解（即八皇后的不同摆放位置的集合）转换成逻辑事实。
3. 再使用 kanren 的逻辑推理引擎 run 函数，从事实 facts 中完成逻辑查询，输出对应的解。

二、算法设计

（一）算法原理

本实验针对八皇后问题设计算法，基于**回溯算法**、**逻辑编程**的原理。

通过递归，逐步尝试，同时进行检验放置是否合理（合理则继续下一步，不合理则返回到上一 步重新尝试），从而达到逐步缩小解空间数量的目的。最终找到所有满足条件的放置方案。

生成所有的放置方案后，利用 kanren 库提供的逻辑推理函数，将所有解转换成 relation 和 facts，再通过 run 进行相应的逻辑查询。

（二）算法功能与设计思路

1. 算法功能

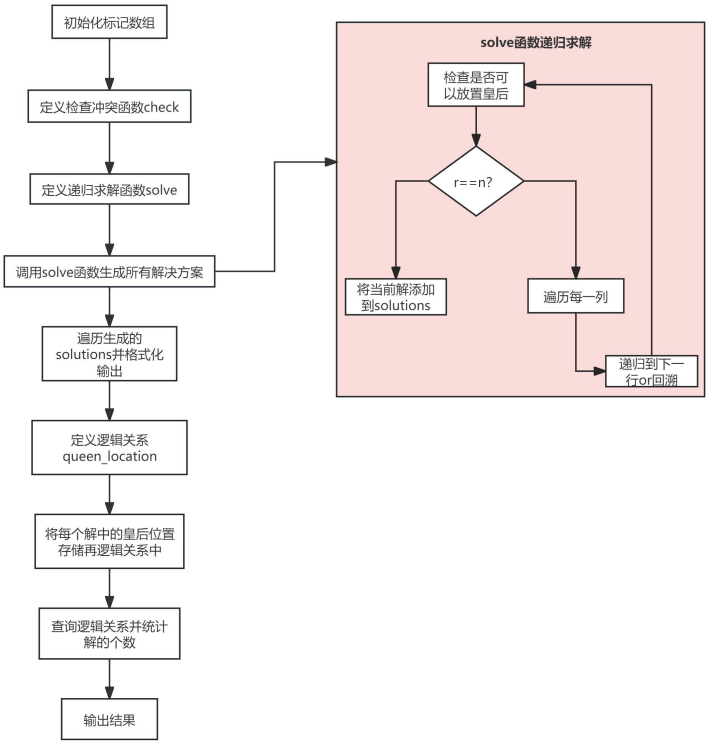
- 检查是否满足约束条件--check(r, c)函数
- 递归求解所有皇后放置方案--solve(r, current_solution)函数等
- 输出放置方案

2. 设计思路

整个算法思路为递归回溯，具体算法设计思路如下：

1. 初始化标记数组：初始化时，所有数组的值均为 0，表示尚未放置皇后；
2. 检查函数：检查第 r 行第 c 列是否可以放置皇后；
3. 递归函数：递归地放置皇后，并找到所有满足条件的解；
4. 定义逻辑关系并查询：对于每个解，将第一个皇后的位置与其他所有皇后的位置组成一个 facts 事实对，从而进行查询。

（三）算法流程图



三、算法实现--本实验在 anaconda 的 spyder 编辑器下进行

（一）实现细节

1. 初始化：

- 初始化列、左上到右下对角线、左下到右上对角线的标记数组
- 定义检查函数 check 和递归求解函数 solve

2. 递归求解：

- Solve 函数递归地尝试在每一行放置皇后
- 如果当前行数 r=n，则表明找到一个解，将当前解添加到 solutions 列表中
- 否则，遍历每一列，检查是否可以放置皇后，如果可以，则递归到下一行并回溯

3. **主函数 queens：**

- 调用 solve 函数生成所有解决方案
- 遍历生成的解决方案，格式化输出并打印解的数量

4. 定义逻辑关系 queen_location

- 使用 kanren 库定义逻辑关系 queen_location
- 将每个解中的第一个皇后位置与其他所有皇后位置组成一个事实对，并存储在逻辑事实 facts 中

5. 查询逻辑关系

- 查询并打印相关的皇后位置，并统计满足条件的解的个数

（二）数据结构定义

1. 列表：

- Col:用于标记列是否有皇后
- Down：用于标记左上到右下的对角线是否有皇后
- Up：用于标记左下到右上的对角线是否有皇后
- Solutions：存储所有可能的解

2. 元组：

- Current_solution:当前递归过程中的解决方案
- Other_queens:除了第一个皇后外的所有皇后的位置

3. 逻辑关系 Relation&facts

- Queen_location:使用 kanren 库定义的逻辑关系

（三）模块的输入输出

1. 输入：

- n:棋盘的大小，即 n 皇后中的 n（n*n 的棋盘）

2. 输出：

- Count：所有可能的解的数量
- 查询与“ai”（第一行第 i 列）相关的其他皇后位置，并打印解的数量

（四）核心函数

1. 检查函数 check(r, c)

```python
#检查r行c列是否可以放置皇后
def check(r, c):
    return col[c] == 0 and down[r + c] == 0 and up[r - c] == 0
```

2. 递归求解函数 solve(r, current_solution)

```python
#递归求解皇后摆放的位置
def solve(r, current_solution):
    solutions = []
    for c in range(n):
        if check(r, c):
            #（r，c）位置放置后，其列、上下对角线均设置为1，禁止其他皇后摆放
            col[c] = down[r + c] = up[r - c] = 1
            current_solution.append(c)

            if r == n - 1:#递归终止条件
                solutions.append(current_solution[:])
            else:#递归方程
                solutions.extend(solve(r + 1, current_solution))

            current_solution.pop()
            col[c] = down[r + c] = up[r - c] = 0
    return solutions
```

3. 主函数 queens(n)

```python
def queens(n):
    #用来标记对角线和列放置皇后的情况 0代表没放，1代表放了
    col = [0] * n
    down = [0] * (n * 2 - 1)#左上到右下的对角线
    up = [0] * (n * 2 - 1)#左下到右上的对角线

    #检查r行c列是否可以放置皇后
    def check(r, c):
```

```python

#递归求解皇后摆放的位置
def solve(r, current_solution):

#queens主函数部分
solutions = []
count=0
# 遍历所有解决方案
for solution in solve(0, []):
    current = []

    # enumerate返回列表里每个元素的索引k和值v
    for k, v in enumerate(solution):
        # 将整数 v 转换为字母-行
        letter = chr(v + ord('a'))
        # 将索引 k 转换为列号（从 1 开始）
        row_number = str(k + 1)
        # 组合字母和列号
        position = letter + row_number
        # 将组合结果添加到 current 列表中
        current.append(position)

    # 对 current 列表进行排序，current是每一个解决方案list
    current.sort()
    # 将排序后的 current 列表添加到 solutions 列表中
    solutions.append(current)
    count=count+1

print(f"共有：{count}个解")
# 返回所有解决方案
return solutions
```

4. 将解转换成逻辑关系并查询

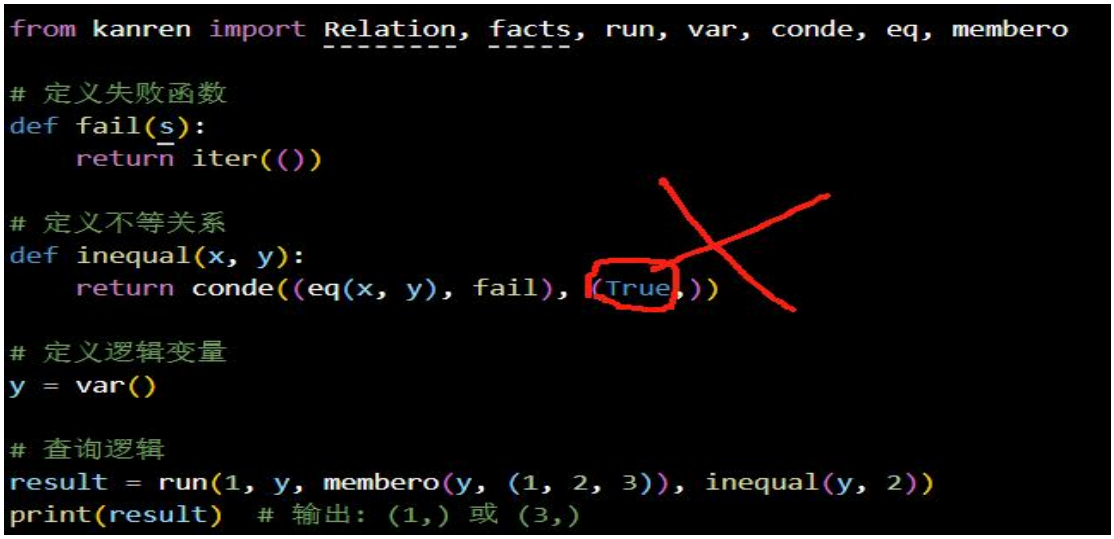
```python
result = queens(8)

# 定义逻辑关系
queen_location = Relation()

# 将每个解中的第一个皇后位置与其他所有皇后位置组成一个事实对
for solution in result:
    first_queen = solution[0]  # 第一个皇后位置
    other_queens = solution[1:]  # 其他所有皇后位置
    facts(queen_location, (first_queen, tuple(other_queens)))  # 将它们组成一个事实对

x = var()
# 查询与 "ai"（i=1，2，3，4，5，6，7，8） 相关的皇后位置
for i in range(1, 9):
    other_locations = run(0, x, queen_location(f"a{i}", x))
    number_of_result = len(other_locations)
    print(f"当第一个皇后放在第1行第{i}列，其他皇后的位置为：")
    print(other_locations)
    print(f"共：{number_of_result}个解")
```

四、实验结果

（一）实验过程（结合调试过程说明）

第一阶段：首先学习了一些 python 的基础语法知识，然后对本次实验要求使用的逻辑库 kanren 进行了学习；

第二阶段：想把八皇后问题的三条约束规则用 kanren 库里的函数表示出来。但是八皇后问题的三条约束规则均涉及的是否定逻辑，而 kanren 库内并没有关于逻辑否定的函数，因此针对逻辑否定的实现进行了大量的尝试。如下图不完全所示：

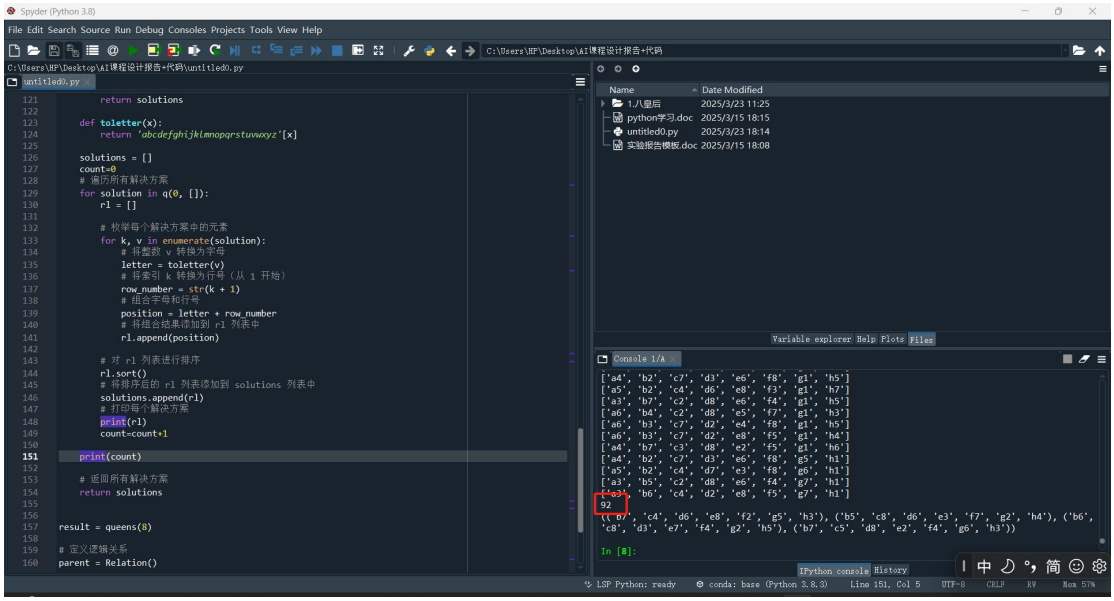


无论是 kanren 库里本身不存在的 neq、lnot 等表示逻辑非的函数，还是自定义的不等关系函数，最终都无法实现。本阶段关于逻辑非无法实现的总结与分析写在第五部分的总结与分析中，在此不再赘述。

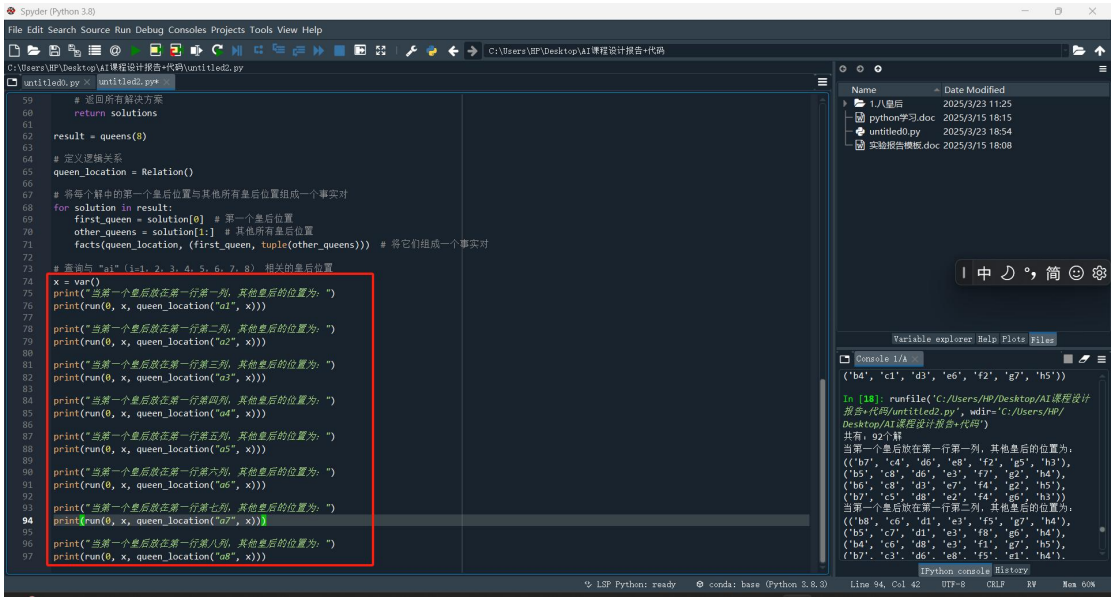
第三阶段：由于约束规则本身无法用逻辑去定义，因此采用递归函数的方式，得到所有解的列表后，转成元组，从而得到逻辑事实，再使用逻辑查询得到结果。在这一阶段，由于本题目的递归函数逻辑十分简单，约束规则也很容易检验，因此不存在对递归函数的调试过程。该阶段的调试主要是基于对得到的递归解的 list 如何转成逻辑 facts 进行了尝试--主要就是将 list 转成 tuple，在将 tuple 放入逻辑 facts 中，最后进行查询。最初，在定义事实

对时，将 other_queens 直接作为列表传递给 facts 函数，导致查询结果不正确。后面发现需要将 other_queens 转换为元组，因为 kanren 库在处理事实对时需要元组类型的数据结构。因此，将 facts 函数中的参数修改为 queen_location(f"a{i}", tuple(other_queens))，就解决了查询结果不正确的问题。

具体的调试过程如下图所示：



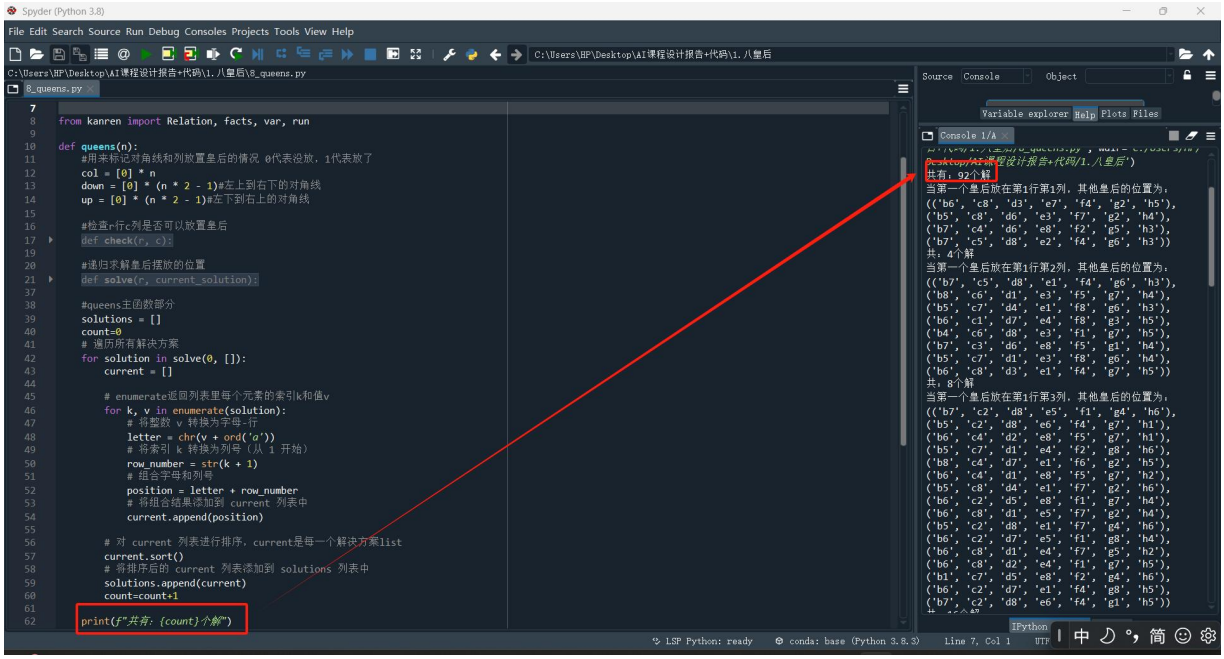
（这里是将得到的解全部输出，便于检验最后逻辑查询的正确性和是否得到了全部解）

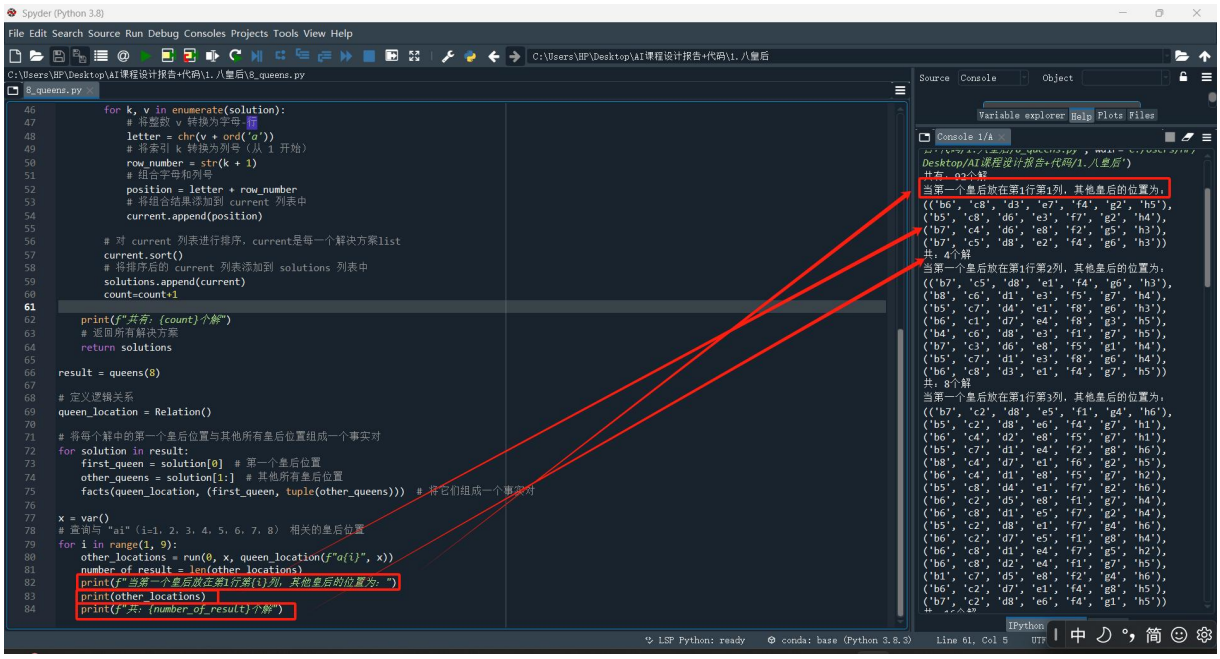


（这里是最初输出所有放置位置的相关皇后位置，后续将该部分融合成了循环，减少了冗余）

除上图所示的调试过程外，在第三阶段的尝试中，还对原有的代码结构进行了优化，确保代码结构清晰，可读性强，代码冗余重复的地方少。

（二）结果展示





上图展示了最终代码输出的结果。最后输出的有：所有解的个数、与“ai”（第一行第 i 列）相关的其他皇后位置、每一个放置位置对应的解的数量。

所有解的结果如下：

```
Console 1/A
Desktop/AI课程设计报告+代码/1.八皇后')
共有：92个解
当第一个皇后放在第1行第1列，其他皇后的位置为：
(('b6', 'c8', 'd3', 'e7', 'f4', 'g2', 'h5'),
('b5', 'c8', 'd6', 'e3', 'f7', 'g2', 'h4'),
('b7', 'c4', 'd6', 'e8', 'f2', 'g5', 'h3'),
('b7', 'c5', 'd8', 'e2', 'f4', 'g6', 'h3'))
共：4个解
当第一个皇后放在第1行第2列，其他皇后的位置为：
(('b5', 'c7', 'd4', 'e1', 'f8', 'g6', 'h3'),
('b8', 'c6', 'd1', 'e3', 'f5', 'g7', 'h4'),
('b6', 'c8', 'd3', 'e1', 'f4', 'g7', 'h5'),
('b7', 'c5', 'd8', 'e1', 'f4', 'g6', 'h3'),
('b4', 'c6', 'd8', 'e3', 'f1', 'g7', 'h5'),
('b6', 'c1', 'd7', 'e4', 'f8', 'g3', 'h5'),
('b5', 'c7', 'd1', 'e3', 'f8', 'g6', 'h4'),
('b7', 'c3', 'd6', 'e8', 'f5', 'g1', 'h4'))
共：8个解
当第一个皇后放在第1行第3列，其他皇后的位置为：
(('b5', 'c8', 'd4', 'e1', 'f7', 'g2', 'h6'),
('b6', 'c8', 'd1', 'e4', 'f7', 'g5', 'h2'),
('b6', 'c2', 'd5', 'e8', 'f1', 'g7', 'h4'),
('b5', 'c2', 'd8', 'e1', 'f7', 'g4', 'h6'),
('b8', 'c4', 'd7', 'e1', 'f6', 'g2', 'h5'),
('b6', 'c8', 'd1', 'e5', 'f7', 'g2', 'h4'),
('b6', 'c2', 'd7', 'e1', 'f4', 'g8', 'h5'),
('b7', 'c2', 'd8', 'e5', 'f1', 'g4', 'h6'),
('b6', 'c8', 'd2', 'e4', 'f1', 'g7', 'h5'),
('b5', 'c2', 'd8', 'e6', 'f4', 'g7', 'h1'),
('b6', 'c4', 'd1', 'e8', 'f5', 'g7', 'h2'),
('b6', 'c4', 'd2', 'e8', 'f5', 'g7', 'h1'),
('b7', 'c2', 'd8', 'e6', 'f4', 'g1', 'h5'),
('b6', 'c2', 'd7', 'e5', 'f1', 'g8', 'h4'),
('b5', 'c7', 'd1', 'e4', 'f2', 'g8', 'h6'),
('b1', 'c7', 'd5', 'e8', 'f2', 'g4', 'h6'))
共：16个解
IPython console History
```

```
Console 1/A
当第一个皇后放在第1行第4列，其他皇后的位置为：
(('b7', 'c5', 'd3', 'e1', 'f6', 'g8', 'h2'),
('b7', 'c5', 'd2', 'e6', 'f1', 'g3', 'h8'),
('b8', 'c1', 'd5', 'e7', 'f2', 'g6', 'h3'),
('b2', 'c5', 'd8', 'e6', 'f1', 'g3', 'h7'),
('b6', 'c8', 'd3', 'e1', 'f7', 'g5', 'h2'),
('b6', 'c1', 'd5', 'e2', 'f8', 'g3', 'h7'),
('b7', 'c1', 'd8', 'e5', 'f2', 'g6', 'h3'),
('b2', 'c8', 'd5', 'e7', 'f1', 'g3', 'h6'),
('b7', 'c3', 'd8', 'e2', 'f5', 'g1', 'h6'),
('b6', 'c8', 'd2', 'e7', 'f1', 'g3', 'h5'),
('b8', 'c1', 'd3', 'e6', 'f2', 'g7', 'h5'),
('b2', 'c7', 'd3', 'e6', 'f8', 'g1', 'h5'),
('b1', 'c5', 'd8', 'e6', 'f3', 'g7', 'h2'),
('b8', 'c5', 'd3', 'e1', 'f7', 'g2', 'h6'),
('b2', 'c7', 'd5', 'e1', 'f8', 'g6', 'h3'),
('b2', 'c8', 'd6', 'e1', 'f3', 'g5', 'h7'),
('b2', 'c7', 'd3', 'e6', 'f8', 'g5', 'h1'),
('b1', 'c5', 'd8', 'e2', 'f7', 'g3', 'h6'))
共：18个解
当第一个皇后放在第1行第5列，其他皇后的位置为：
(('b8', 'c4', 'd1', 'e3', 'f6', 'g2', 'h7'),
('b7', 'c2', 'd4', 'e8', 'f1', 'g3', 'h6'),
('b7', 'c1', 'd3', 'e8', 'f6', 'g4', 'h2'),
('b2', 'c4', 'd7', 'e3', 'f8', 'g6', 'h1'),
('b1', 'c8', 'd6', 'e3', 'f7', 'g2', 'h4'),
('b7', 'c2', 'd6', 'e3', 'f1', 'g8', 'h4'),
('b3', 'c1', 'd7', 'e2', 'f8', 'g6', 'h4'),
('b2', 'c8', 'd1', 'e4', 'f7', 'g3', 'h6'),
('b2', 'c6', 'd1', 'e7', 'f4', 'g8', 'h3'),
('b7', 'c4', 'd1', 'e3', 'f8', 'g6', 'h2'),
('b7', 'c2', 'd6', 'e3', 'f1', 'g4', 'h8'),
('b8', 'c4', 'd1', 'e7', 'f2', 'g6', 'h3'),
('b3', 'c8', 'd4', 'e7', 'f1', 'g6', 'h2'),
('b3', 'c1', 'd6', 'e8', 'f2', 'g4', 'h7'),
('b7', 'c1', 'd4', 'e2', 'f8', 'g6', 'h3'),
IPython console History
```

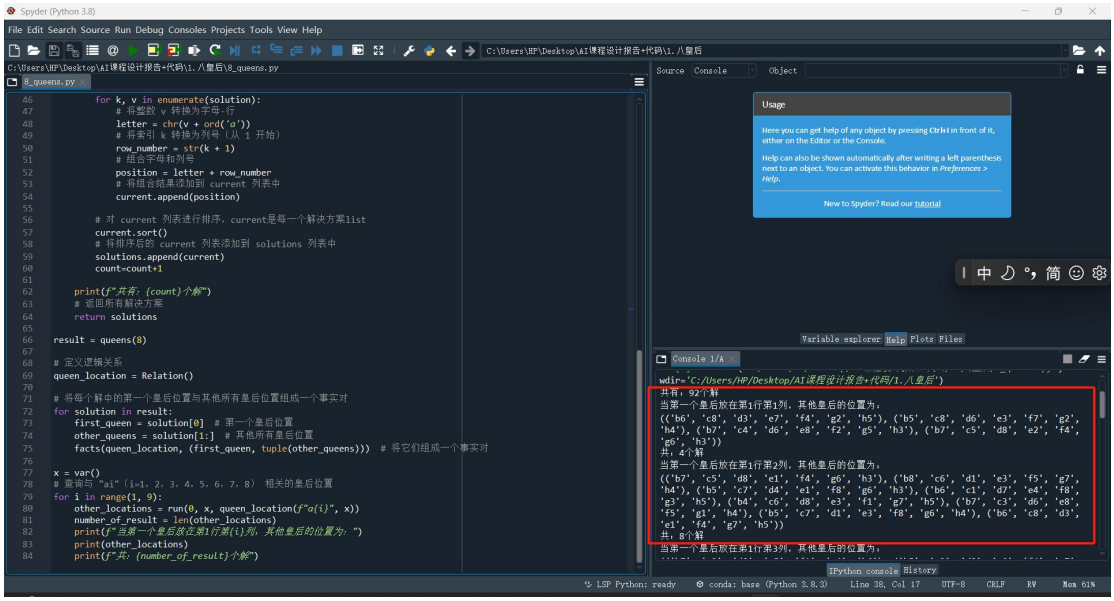
```
Console 1/A
('b1', 'c4', 'd6', 'e8', 'f2', 'g7', 'h3'),
('b1', 'c8', 'd4', 'e2', 'f7', 'g3', 'h6'),
('b2', 'c4', 'd6', 'e8', 'f3', 'g1', 'h7'))
共：18个解
当第一个皇后放在第1行第6列，其他皇后的位置为：
(('b8', 'c2', 'd4', 'e1', 'f7', 'g5', 'h3'),
('b3', 'c1', 'd8', 'e4', 'f2', 'g7', 'h5'),
('b4', 'c7', 'd1', 'e3', 'f5', 'g2', 'h8'),
('b3', 'c5', 'd7', 'e1', 'f4', 'g2', 'h8'),
('b3', 'c1', 'd7', 'e5', 'f8', 'g2', 'h4'),
('b3', 'c1', 'd8', 'e5', 'f2', 'g4', 'h7'),
('b2', 'c7', 'd1', 'e3', 'f5', 'g8', 'h4'),
('b3', 'c5', 'd8', 'e1', 'f4', 'g2', 'h7'),
('b4', 'c2', 'd8', 'e5', 'f7', 'g1', 'h3'),
('b3', 'c7', 'd2', 'e8', 'f5', 'g1', 'h4'),
('b2', 'c7', 'd1', 'e4', 'f8', 'g5', 'h3'),
('b4', 'c1', 'd5', 'e8', 'f2', 'g7', 'h3'),
('b4', 'c7', 'd1', 'e8', 'f2', 'g5', 'h3'),
('b3', 'c7', 'd2', 'e4', 'f8', 'g1', 'h5'),
('b3', 'c7', 'd4', 'e1', 'f8', 'g2', 'h5'),
('b1', 'c5', 'd2', 'e8', 'f3', 'g7', 'h4'))
共：16个解
当第一个皇后放在第1行第7列，其他皇后的位置为：
(('b1', 'c3', 'd8', 'e6', 'f4', 'g2', 'h5'),
('b2', 'c6', 'd3', 'e1', 'f4', 'g8', 'h5'),
('b3', 'c1', 'd6', 'e8', 'f5', 'g2', 'h4'),
('b2', 'c4', 'd1', 'e8', 'f5', 'g3', 'h6'),
('b3', 'c8', 'd2', 'e5', 'f1', 'g6', 'h4'),
('b4', 'c2', 'd5', 'e8', 'f1', 'g3', 'h6'),
('b4', 'c2', 'd8', 'e6', 'f1', 'g3', 'h5'),
('b5', 'c3', 'd1', 'e6', 'f8', 'g2', 'h4'))
共：8个解
当第一个皇后放在第1行第8列，其他皇后的位置为：
(('b3', 'c1', 'd6', 'e2', 'f5', 'g7', 'h4'),
('b2', 'c4', 'd1', 'e7', 'f5', 'g3', 'h6'),
('b4', 'c1', 'd3', 'e6', 'f2', 'g7', 'h5'),
IPython console History
```

```
Console 1/A
('b2', 'c5', 'd3', 'e1', 'f7', 'g4', 'h6'))
共：4个解

In [2]:
IPython console History
```

（三）结果分析

“结果展示”中的第一幅图显示的是所有可能的解的个数；第二幅图则是具体逻辑查询的结果，含义为：当第 1 行第 i 列的皇后摆放完成后，与它互不攻击的其他七个皇后的位置（‘b’代表第 2 行，其余字母以此类推）以及在该摆放方式下共有几种方式。

注意：由于逻辑推理引擎 run 的返回值是一个元组，本题目输出并没有将这个元组进行格式化的输出，因此只有调整终端到合适位置才能有如图所示的输出格式。若未调节，则可能如下图所示：



五、总结与分析

（一）问题与解决方案

1. 关于 membero 的使用：

membero 是来限制一个逻辑变量 var 的取值范围，但是在 membero 的调试中发现，membero 对逻辑变量取值范围的约束只能在 run 函数中被显式地使用，才会对查询结果产生影响。

```
x=var()
membero(x,("b","a","l"))

result=run 1,x,parent(x,"b"))

print(result) 为什么我指明x属于一个没有h的集合，最后的result结果还是h?
```

你的问题非常有洞察力！这个问题的关键在于理解 Kanren 的逻辑编程机制，特别是逻辑变量的绑定和目标构造器的作用。让我们逐步分析代码和逻辑，以解释为什么最终结果仍然是 `h` 。

代码分析

1. 定义关系和事实

Python

```python
parent = Relation()
facts(parent, ("h", "b"), ("h", "l"), ("a", "h"))
```

所以membero的约束只能在run（）里面使用才会对查询结果有约束吗？ 那membero存在的意义是什么?

你的问题非常关键，触及了 Kanren 中逻辑关系和约束的核心概念。让我们深入探讨 `membero` 的作用以及它如何在逻辑编程中发挥作用。

- `membero` **的约束必须在** `run` **中使用**，才能对查询结果产生影响。
- `membero` **的存在意义**在于提供声明性约束，允许逻辑变量属于某个集合 并与其他逻辑关系组合，形成复杂的约束条件。
- **逻辑编程的核心**是通过声明逻辑关系，让 `run` 函数来求解逻辑变量的值。

2. 不等关系的表示：

前文中已经提到，在解决该问题的时候，最初的想法是想将规则约束，用逻辑库中的关系和事实定义出来，但是后面发现几乎是无法实现的。关于此问题，我和同学、gpt 还有老师进行了探讨，也尝试了提出的所有方法--使用 kanren 库中的 neq、lnot 函数（最后发现 kanren 库的源代码中根本没有这些函数）、自定义不等关系（根本不起作用）等。最终我个人认为，kanren 库本身无法实现逻辑否定的规则约束（除非将所有否定的情况罗列出来，但这种方法太愚蠢了），因为考虑到了逻辑否定本身涉及变量的自由状态和闭世界假设，想要实现非常复杂，而且 run 函数本身具有惰性，不会对解空间进行多次尝试来寻找到满足不等关系的解，因此最后采用递归求解的方式。

3. 逻辑变量 var（）能否进行算术运算：

在尝试定义不等关系的时候遇到了另一个有趣的问题。Kanren 库涉及的逻辑变量 var，即使可以通过 eq 函数对 var 进行赋值，但是 var 本身无法直接与各种形式的数字进行算术运算，需要通过定义函数“实现”这种算术运算。

```
#print(result_list)  # 输出结果
from kanren import run, var, fact
from kanren.assoccomm import eq_assoccomm as eq
from kanren.assoccomm import commutative, associative

# Define some dummy Operationss
add = 'add'
mul = 'mul'
# Declare that these ops are commutative using the facts system
fact(commutative, mul)
fact(commutative, add)
fact(associative, mul)
fact(associative, add)

# Define some wild variables
x, y = var('x'), var('y')

# Two expressions to match
pattern = (mul, (add, 1, x), y)                # (1 + x) * y
expr    = (mul, 2, (add, 3, 1))                # 2 * (3 + 1)
print(run(0, (x,y), eq(pattern, expr)))        # prints ((3, 2),) meaning
                                               #   x matches to 3
                                               #   y matches to 2
```

（本人进行的一些尝试，确实可以实现加和乘运算）

（二）收获与思考

其实在本次实验中，我思考最多、收获最大的地方就是关于不等关系，即逻辑否定的定义。在得出“不等关系无法用 kanren 库实现”的结论之前，我和同学、老师进行了交流，越发确定了这一结论（虽然也不知道该结论是否正确）。在交流过程中，我了解到 kanren 库里的 run 这个求解器本身是惰性的，而不等关系是需要在解空间中不断尝试的，并且 kanren 遵循模式匹配的原则，如果定义不等关系，那得把所有不等的情况全都定义出来（非常愚蠢）。

本次实验不仅让我深入理解了逻辑编程的基本概念和实现方法，还促使我思考如何在其他编程范式中处理类似的问题。尽管逻辑编程在处理某些问题时存在一定的局限性，但它的声明性编程方式和自动求解器特性提供了独特的优势。

（三）算法评价

该算法主要是采用了一个递归函数。因此该算法的优缺点也十分明显，优点就是结构非常清晰易懂，缺点就是递归函数固有的--即算法在处理大规模问题（如 N 非常大时）时

可能面临时间和空间复杂度较高、运行时间过长、栈溢出等问题。

因此，该算法的适用场景即适用于解决规模较小的 n 皇后问题（本次实验的八皇后问题是没有问题的）

六、参考文献（资料）

- https://github.com/logpy/logpy
- https://zh.wikipedia.org/wiki/%E5%85%AB%E7%9A%87%E5%90%8E%E9%97%AE%E9%A2%98
- https://github.com/yousetme/ykanren
- https://github.com/theSherwood/differential_dataflow/commit/0e83001174224d0cb53dd6b23453b85281a66713